

Chapter 5

Algorithm Complexity

- Big O: Worse case time, worse data arrangement / branching
- Big Omega - Best case time, perfect data
- Big Theta - Average case time, data provides average amount of branching and other aspects which slow it down
- Computational complexity: Steps to complete algorithm, called "Time complexity" when factoring in real time

Performance models use real measurements, vs. algorithm complexity which deals with constant time arithmetic terms which may not be constant time in reality due to the actual computer architecture

- Example: Bisection search vs. Linear search: Bisection is better in theory, but caching may make linear search equal or even better in some cases

Parallel Algorithms:

- Break the work up and combine results

Hash function:

- Convert some value to a unique hash which can be computed without communication
- Load factor: When using for parallel methods, this is how much of the problem ends up in each "hash".
 - Load factors above .8 or .9 are ok, above that collisions
- Collision: When two hashes correspond to one object, non-ideal but sometimes required
- Spatial Hash: Break problem up in space, as long as the hash function is communicated, all processes know what space belongs to which process

Case study in chapter for neighbors base on hashing

- Depending on method, up to 1200X speedup can be achieved using hashing rather than tree structure!

Table lookups show similar results, but at a 1,300,000 speedup when using GPU to hash things

Sorting with hashes on GPU is 246X faster than a quicksort on CPU

Summing Methods in Parallel

- Order you sum in affects result
- Small + Big = Loss of precision
- Summing all numbers also known as a sum reduction of array

Methods

- Long-double: Basically the default simple method if trying to maintain precision, may be sensitive to order of array
- Pairwise summation: Just sum in pairs of numbers, combine results in pairs. Order may be bad!
- Kahan Summation: Good when accumulator big number relative to whats being added
 - Uses corrective terms which require 4FLOPS, but cheap due to cache
- Knuth summation: Good when either number may be the big one

- Costs 7 flops per sum, but cache may still help here. Safest technique presented in chapter which is still fast
- Quad-precision Summation: Best precision, order of summation unlikely to matter, simple
 - Quad-precision is often software simulated, may be incredibly slow and unusable of GPUs for example

Future

- Locality: Keep values local to processor, only deal with values which are kept local to minimize communication
 - Locality to Cache: Use values in cache as much as possible, memory is slow
- Async: Do things in parallel, do not coordinate between threads when possible
- Conditionals: Avoid, branching can be inefficient
- Reproducible: Safe methods to maintain precision and results come back reliably
- Arithmetic Intensity: FLOPS may be cheaper than memory, recalculate values sometimes rather than look them up when not in cache